Q1. Consider the following code written in C-like syntax:

```
int i = 1;
int g(void) {
    if (i != 1)
        i=3;
    return i;
}
int f(void) {
    int i=2;
    return g();
}
int main(void) {
    printf("%d", f());
}
```

What will be the output?

(10pts) a) if static scoping is used? 1

(10pts) b) if dynamic scoping is used? 3

(15pts) Q2. Considering your project, what will be the output for the following input:

```
(define x 2) (+ x 3)) (+ x '1') ~let x 2
```

Hint: The tokens in the project are:

LEFTPAR, RIGHTPAR,

LEFTSQUAREB, RIGHTSQUAREB,

LEFTCURLYB, RIGHTCURLYB,

NUMBER, BOOLEAN, CHAR, STRING,

DEFINE, LET, COND, IF, BEGIN, IDENTIFIER

LEFTPAR DEFINE IDENTIFIER NUMBER RIGHTPAR

LEFTPAR IDENTIFIER IDENTIFIER NUMBER RIGHTPAR

RIGHTPAR LEFTPAR IDENTIFIER IDENTIFIER CHAR

RIGHTPAR – row and column numbers

(10pts) Q3. Consider the following declaration:

```
char arr[M][N];
```

Assume that the address of `arr[0][0] = 100` (decimal). Calculate the address of array element `arr[10][2]` if the array is stored in column major order.

ADDRESS= (100+ (2*M + 10)*1)byte

Q4. Consider the following program.

```
int f (int *c) {
   (*c)++;
   return *c;
}
int main() {
   int a=0,b=-2;
   if (a && f(&b)) {
      printf("if \n"); a--;
   }
   else if (a || f(&b)){
      printf("else if \n"); a++;
   }
   else{
       printf("else \n"); b++;
   }
   printf("a= %d b= %d\n", a,b);
   return 0;
}
```

What will be the output, if the language

a) (10pts) uses short-circuit evaluation

else if

a= 1 b= -1

b) (10pts) doesn't use short-circuit evaluation

else

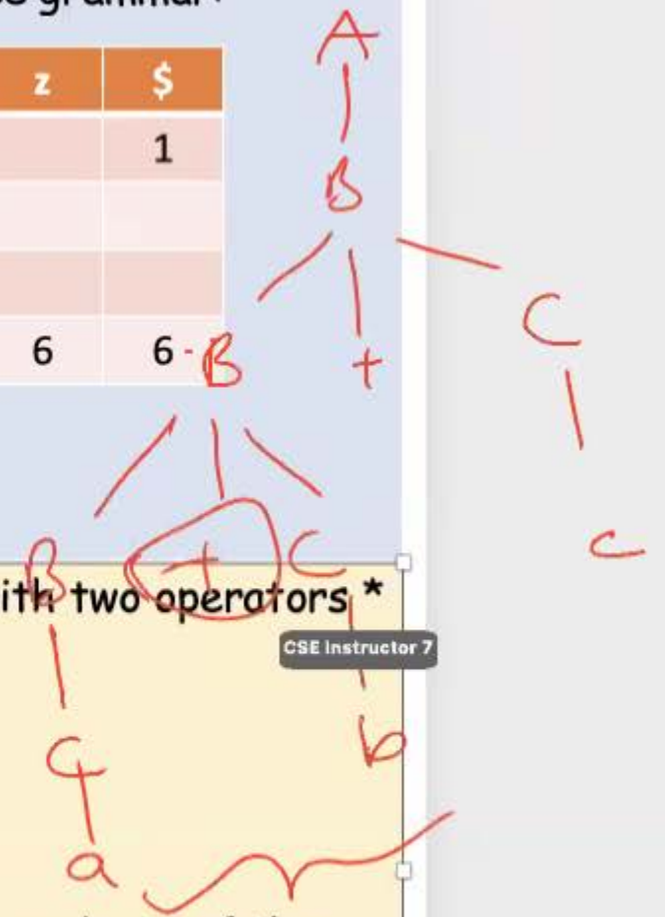
a= 0 b= 1

(15pts) Q5. Consider the following context free grammar:

1. S → AC$
2. A → xAB
3. A → ε
4. B → y
5. C → BCz
6. C → ε

	x	y	z	$
S	1	1		1
A	2	3		
B		4		
C		5	6	6

Give the LL(1) parse table for the grammar.



(20pts) Q6. Consider the following grammar with two operators * and +:

A → B | B * A
B → C | B + C
C → a | b | c

Discuss about the associativity and relative precedence of the operators * and +.

a + b + c

+ > * precedence
* → right assoc + → left assoc